

Homework 01 Solutions

Applied Statistics for Engineers and Scientists II

Ho Huu Binh

05 June 2026

Contents

1. Exercise 1 solution: centering, intercept geometry, and affine regression	1
1.1 Intuition	1
1.2 Definitions	2
1.3 Part (a): affine versus linear	2
1.4 Part (b): decomposition and orthogonality	3
1.5 Part (c): least squares with intercept equals centered least squares plus a shift	4
1.6 Part (d): specialization to simple regression	5
2. Exercise 2 solution: Gram matrix, eigenvalues, condition number, and invertibility	5
2.1 Intuition	5
2.2 Part A: proof of the rank–Gram equivalence	6
2.3 Part B: exact multicollinearity	6
2.4 Part C: approximate multicollinearity	7
3. Exercise 3 solution: multiple regression analysis in R	10
3.1 Part A: tibble and five basic dplyr verbs	10
3.2 Part B: visual exploration with ggplot2	13
3.3 Part C: fit and interpret linear regression models	14
3.4 Part D: residual diagnostics	16
3.5 Part E: Gram matrix and condition number for the car model	18
4. Some remarks	20

```
library(tidyverse)
```

1. Exercise 1 solution: centering, intercept geometry, and affine regression

1.1 Intuition

The intercept changes the geometry of regression. Without an intercept, the fitted function must pass through the origin of the coordinate system. With an intercept, the fitted function is allowed

to shift vertically. Thus a model of the form

$$z \mapsto a + z^\top g$$

is not a purely linear function of the predictor vector z ; it is a shifted linear function. This is why it is called **affine**.

The key fact is that the intercept separates the average level of the response from the slope estimation problem. After subtracting sample means from both the response and predictors, the slope is obtained by fitting a no-intercept regression to the centered data. The intercept is then recovered by shifting the fitted hyperplane back so that it passes through the sample mean point.

1.2 Definitions

Let $Y \in \mathbb{R}^n$ be the response vector and let $Z \in \mathbb{R}^{n \times p}$ be the matrix of non-intercept predictors. Let

$$1_n = (1, \dots, 1)^\top \in \mathbb{R}^n.$$

Define the sample mean of the response and the sample mean vector of the predictors by

$$\bar{Y} = \frac{1}{n} 1_n^\top Y, \quad \bar{z} = \frac{1}{n} Z^\top 1_n \in \mathbb{R}^p.$$

Define the centered response and centered design by

$$Y_c = Y - \bar{Y} 1_n, \quad Z_c = Z - 1_n \bar{z}^\top.$$

Equivalently, the i th row of Z_c is $z_i^\top - \bar{z}^\top$, where $z_i^\top$ is the i th row of Z . Assume

$$\text{rank}(Z_c) = p.$$

This assumption is needed to make $Z_c^\top Z_c$ invertible and hence to make the centered least-squares slope unique.

1.3 Part (a): affine versus linear

A function $f : \mathbb{R}^p \rightarrow \mathbb{R}$ is linear if it preserves addition and scalar multiplication:

$$f(u + v) = f(u) + f(v), \quad f(cu) = cf(u).$$

In particular, every linear function satisfies $f(0) = 0$, because

$$f(0) = f(0 + 0) = f(0) + f(0),$$

so subtracting $f(0)$ from both sides gives $f(0) = 0$.

The map

$$f(z) = a + z^\top g$$

satisfies

$$f(0) = a.$$

Therefore, if $a \neq 0$, this function does not pass through the origin and cannot be linear. It is a translate of the linear map $z \mapsto z^\top g$. Hence it is called affine. In simple regression, an affine model is a line in the plane; in multiple regression, it is a plane or hyperplane in a higher-dimensional coordinate system.

1.4 Part (b): decomposition and orthogonality

We prove the identity

$$Y - a1_n - Zg = (Y_c - Z_cg) + (\bar{Y} - a - \bar{z}^\top g)1_n.$$

By the definitions of Y_c and Z_c ,

$$Y = Y_c + \bar{Y}1_n, \quad Z = Z_c + 1_n\bar{z}^\top.$$

Substitute these two identities into $Y - a1_n - Zg$:

$$\begin{aligned} Y - a1_n - Zg &= (Y_c + \bar{Y}1_n) - a1_n - (Z_c + 1_n\bar{z}^\top)g \\ &= Y_c + \bar{Y}1_n - a1_n - Z_cg - 1_n\bar{z}^\top g \\ &= (Y_c - Z_cg) + (\bar{Y} - a - \bar{z}^\top g)1_n. \end{aligned}$$

This proves the identity.

Next we prove that the two terms on the right-hand side are orthogonal. First,

$$1_n^\top Y_c = 1_n^\top (Y - \bar{Y}1_n) = 1_n^\top Y - \bar{Y}1_n^\top 1_n.$$

Since $1_n^\top 1_n = n$ and $\bar{Y} = n^{-1}1_n^\top Y$,

$$1_n^\top Y_c = 1_n^\top Y - n \left(\frac{1}{n} 1_n^\top Y \right) = 0.$$

Similarly,

$$\begin{aligned} 1_n^\top Z_c &= 1_n^\top (Z - 1_n\bar{z}^\top) \\ &= 1_n^\top Z - 1_n^\top 1_n\bar{z}^\top \\ &= 1_n^\top Z - n\bar{z}^\top. \end{aligned}$$

Because $\bar{z} = n^{-1}Z^\top 1_n$, we have $\bar{z}^\top = n^{-1}1_n^\top Z$. Therefore

$$n\bar{z}^\top = 1_n^\top Z,$$

and hence

$$1_n^\top Z_c = 0.$$

Thus, for every $g \in \mathbb{R}^p$,

$$1_n^\top (Y_c - Z_cg) = 1_n^\top Y_c - 1_n^\top Z_cg = 0 - 0 = 0.$$

Now compute the Euclidean inner product between the two terms:

$$\begin{aligned} \langle Y_c - Z_cg, (\bar{Y} - a - \bar{z}^\top g)1_n \rangle &= (\bar{Y} - a - \bar{z}^\top g)(Y_c - Z_cg)^\top 1_n \\ &= (\bar{Y} - a - \bar{z}^\top g)1_n^\top (Y_c - Z_cg) \\ &= (\bar{Y} - a - \bar{z}^\top g) \cdot 0 \\ &= 0. \end{aligned}$$

Therefore $Y_c - Z_cg$ is orthogonal to $(\bar{Y} - a - \bar{z}^\top g)1_n$.

1.5 Part (c): least squares with intercept equals centered least squares plus a shift

By the decomposition and orthogonality proved above,

$$Y - a1_n - Zg = (Y_c - Z_c g) + (\bar{Y} - a - \bar{z}^\top g)1_n,$$

with the two summands orthogonal. Therefore the squared norm is the sum of squared norms:

$$\|Y - a1_n - Zg\|_2^2 = \|Y_c - Z_c g\|_2^2 + \|(\bar{Y} - a - \bar{z}^\top g)1_n\|_2^2.$$

The second term is

$$\begin{aligned} \|(\bar{Y} - a - \bar{z}^\top g)1_n\|_2^2 &= (\bar{Y} - a - \bar{z}^\top g)^2 \|1_n\|_2^2 \\ &= (\bar{Y} - a - \bar{z}^\top g)^2 n. \end{aligned}$$

Thus

$$\|Y - a1_n - Zg\|_2^2 = \|Y_c - Z_c g\|_2^2 + n(\bar{Y} - a - \bar{z}^\top g)^2.$$

For a fixed g , the first term $\|Y_c - Z_c g\|_2^2$ does not depend on a . The second term is a nonnegative quadratic function of a :

$$n(\bar{Y} - a - \bar{z}^\top g)^2.$$

It is minimized if and only if the square is zero, namely when

$$\bar{Y} - a - \bar{z}^\top g = 0.$$

Solving for a gives

$$a = \bar{Y} - \bar{z}^\top g.$$

Therefore, after minimizing over a , the original problem reduces to

$$\min_{g \in \mathbb{R}^p} \|Y_c - Z_c g\|_2^2.$$

Define

$$Q_c(g) = \|Y_c - Z_c g\|_2^2.$$

Expanding,

$$\begin{aligned} Q_c(g) &= (Y_c - Z_c g)^\top (Y_c - Z_c g) \\ &= Y_c^\top Y_c - 2g^\top Z_c^\top Y_c + g^\top Z_c^\top Z_c g. \end{aligned}$$

The gradient is

$$\nabla Q_c(g) = -2Z_c^\top Y_c + 2Z_c^\top Z_c g,$$

and the Hessian is

$$\nabla^2 Q_c(g) = 2Z_c^\top Z_c.$$

Because $\text{rank}(Z_c) = p$, for every nonzero $v \in \mathbb{R}^p$,

$$v^\top Z_c^\top Z_c v = (Z_c v)^\top (Z_c v) = \|Z_c v\|_2^2 > 0.$$

Hence $Z_c^\top Z_c$ is positive definite and invertible. Consequently Q_c is strictly convex and has a unique minimizer. Setting the gradient equal to zero gives

$$Z_c^\top Z_c g = Z_c^\top Y_c.$$

Multiplying by $(Z_c^\top Z_c)^{-1}$, we obtain

$$\hat{\gamma} = (Z_c^\top Z_c)^{-1} Z_c^\top Y_c.$$

The corresponding intercept is

$$\hat{\alpha} = \bar{Y} - \bar{z}^\top \hat{\gamma}.$$

Finally, the fitted vector is

$$\begin{aligned} \hat{Y} &= \hat{\alpha} 1_n + Z \hat{\gamma} \\ &= (\bar{Y} - \bar{z}^\top \hat{\gamma}) 1_n + Z \hat{\gamma} \\ &= \bar{Y} 1_n + (Z - 1_n \bar{z}^\top) \hat{\gamma} \\ &= \bar{Y} 1_n + Z_c \hat{\gamma}. \end{aligned}$$

Thus fitting with an intercept is exactly equivalent to fitting the centered response on the centered predictors without an intercept, and then shifting the fitted vector back by $\bar{Y} 1_n$.

1.6 Part (d): specialization to simple regression

In simple regression, $p = 1$, $Z = x \in \mathbb{R}^n$, $\bar{z} = \bar{x}$, and $Z_c = x_c$, where

$$x_c = (x_1 - \bar{x}, \dots, x_n - \bar{x})^\top.$$

The fitted value at a scalar point x is

$$\hat{y}(x) = \hat{\alpha} + \hat{\gamma}x.$$

From the formula above,

$$\hat{\alpha} = \bar{Y} - \bar{x} \hat{\gamma}.$$

Therefore

$$\begin{aligned} \hat{y}(\bar{x}) &= \hat{\alpha} + \hat{\gamma} \bar{x} \\ &= (\bar{Y} - \bar{x} \hat{\gamma}) + \hat{\gamma} \bar{x} \\ &= \bar{Y}. \end{aligned}$$

Thus the least-squares line with an intercept passes through the sample centroid $(\bar{x}, \bar{Y})$. Geometrically, the centered fit estimates a line through the origin in the centered coordinate system $(x_i - \bar{x}, Y_i - \bar{Y})$. Translating this line back to the original coordinate system forces the fitted line to pass through the original centroid.

2. Exercise 2 solution: Gram matrix, eigenvalues, condition number, and invertibility

2.1 Intuition

The Gram matrix

$$G = X^\top X$$

contains the inner products among the columns of X . If one column of X is exactly a linear combination of the other columns, then X has redundant information and G is singular. In that case $(X^\top X)^{-1}$ does not exist, so the ordinary least-squares formula cannot be used.

If the columns are not exactly dependent but are nearly dependent, then G is invertible but has a very small eigenvalue. A small eigenvalue means that there is a direction in coefficient space along which the fitted vector changes very little. Consequently, many different coefficient vectors can produce almost the same fitted values, and individual coefficients become unstable.

2.2 Part A: proof of the rank–Gram equivalence

Let $X \in \mathbb{R}^{n \times q}$. We prove

$$X^\top X \text{ is invertible} \iff \text{rank}(X) = q.$$

For every $v \in \mathbb{R}^q$,

$$\begin{aligned} v^\top X^\top X v &= (Xv)^\top (Xv) \\ &= \|Xv\|_2^2. \end{aligned}$$

First suppose $\text{rank}(X) = q$. This means the columns of X are linearly independent. Therefore, if $v \neq 0$, then $Xv \neq 0$. Hence

$$v^\top X^\top X v = \|Xv\|_2^2 > 0 \quad \text{for all } v \neq 0.$$

Thus $X^\top X$ is positive definite. A positive definite matrix has no nonzero vector in its null space: if $X^\top X v = 0$, then

$$v^\top X^\top X v = v^\top 0 = 0,$$

which contradicts positive definiteness unless $v = 0$. Hence the null space of $X^\top X$ is $\{0\}$, so $X^\top X$ is invertible.

Conversely, suppose $\text{rank}(X) < q$. Then the columns of X are linearly dependent. Therefore there exists a nonzero vector $v \in \mathbb{R}^q$ such that

$$Xv = 0.$$

Multiplying by $X^\top$ gives

$$X^\top X v = X^\top 0 = 0.$$

Thus $v \neq 0$ lies in the null space of $X^\top X$. Hence $X^\top X$ is not invertible.

Combining the two implications proves the equivalence.

2.3 Part B: exact multicollinearity

Consider

$$X_{\text{exact}} = \begin{pmatrix} 1 & 1 & 2 \\ 1 & 2 & 4 \\ 1 & 3 & 6 \\ 1 & 4 & 8 \end{pmatrix}.$$

The second column is $(1, 2, 3, 4)^\top$. The third column is $(2, 4, 6, 8)^\top$, which equals twice the second column. Hence the columns are linearly dependent. Therefore

$$\text{rank}(X_{\text{exact}}) < 3.$$

The first and second columns are not multiples of each other, so the rank is exactly 2. By the result proved above, $X_{\text{exact}}^\top X_{\text{exact}}$ is not invertible.

```

X_exact <- matrix(
  c(1, 1, 1, 1,
    1, 2, 3, 4,
    2, 4, 6, 8),
  nrow = 4,
  ncol = 3,
  byrow = FALSE
)

G_exact <- crossprod(X_exact)
lambda_exact <- eigen(G_exact, symmetric = TRUE)$values
rank_exact <- qr(X_exact)$rank
det_exact <- det(G_exact)
kappa_exact <- condition_number_from_eigenvalues(lambda_exact)

```

```
G_exact
```

```
##      [,1] [,2] [,3]
## [1,]    4   10   20
## [2,]   10   30   60
## [3,]   20   60  120
```

```
lambda_exact
```

```
## [1] 1.533479e+02 6.521120e-01 -2.842171e-14
```

```
rank_exact
```

```
## [1] 2
```

```
det_exact
```

```
## [1] 0
```

```
kappa_exact
```

```
## [1] Inf
```

The Gram matrix is

$$G_{\text{exact}} = \begin{pmatrix} 4 & 10 & 20 \\ 10 & 30 & 60 \\ 20 & 60 & 120 \end{pmatrix}.$$

Its smallest eigenvalue is zero in exact mathematics. Floating-point computation may return a very small number near zero, because numerical eigenvalue algorithms work with finite precision. The determinant is zero, and the condition number is infinite. Statistically, this means the ordinary least-squares coefficients are not uniquely identified: increasing the coefficient of the second column and decreasing the coefficient of the third column in a compensating way can leave the fitted values unchanged.

2.4 Part C: approximate multicollinearity

Now use the approximately collinear dataset.

```
approx_data <- tibble(
  x1 = 1:10,
  x2 = x1 + c(0.02, -0.01, 0.03, -0.02, 0.01,
              -0.03, 0.02, -0.01, 0.01, -0.02),
  y = c(2.2, 2.9, 4.1, 4.8, 6.2, 6.9, 8.1, 8.8, 10.1, 10.9)
)
```

```
approx_data
```

```
## # A tibble: 10 x 3
##       x1     x2     y
##   <int> <dbl> <dbl>
## 1     1     1.02  2.2
## 2     2     1.99  2.9
## 3     3     3.03  4.1
## 4     4     3.98  4.8
## 5     5     5.01  6.2
## 6     6     5.97  6.9
## 7     7     7.02  8.1
## 8     8     7.99  8.8
## 9     9     9.01 10.1
## 10    10     9.98 10.9
```

For the model

$$y_i = \beta_0 + \beta_1 x_{i1} + \beta_2 x_{i2} + \varepsilon_i,$$

the design matrix contains an intercept column, the x1 column, and the x2 column.

```
X_approx <- model.matrix(y ~ x1 + x2, data = approx_data)
G_approx <- crossprod(X_approx)
lambda_approx <- eigen(G_approx, symmetric = TRUE)$values
condition_approx <- condition_number_from_eigenvalues(lambda_approx)
rank_approx <- qr(X_approx)$rank
```

```
G_approx
```

```
##           (Intercept)      x1      x2
## (Intercept)          10 55.00 55.0000
## x1                   55 385.00 384.8300
## x2                   55 384.83 384.6638
```

```
lambda_approx
```

```
## [1] 7.775442e+02 2.117853e+00 1.728279e-03
```

```
condition_approx
```

```
## [1] 449895
```

```
rank_approx
```

```
## [1] 3
```


The rank is full, so the Gram matrix is invertible. However, the condition number is very large. This indicates that the columns are almost linearly dependent. In this situation, the fitted values may still be stable, but the individual coefficient estimates may be unstable.

We now change only one response value by 0.1 and compare the coefficients.

```
fit_original <- lm(y ~ x1 + x2, data = approx_data)

approx_data_perturbed <- approx_data |>
  mutate(y = if_else(row_number() == n(), y + 0.1, y))

fit_perturbed <- lm(y ~ x1 + x2, data = approx_data_perturbed)

coef_original <- coef(fit_original)
coef_perturbed <- coef(fit_perturbed)
coef_change <- coef_perturbed - coef_original

coef_original
```

```
## (Intercept)          x1          x2
##    1.001933   -5.300422    6.300070
```

```
coef_perturbed
```

```
## (Intercept)          x1          x2
##    0.9854568  -4.9846451    5.9891075
```

```
coef_change
```

```
## (Intercept)          x1          x2
## -0.01647576  0.31577653 -0.31096275
```

Although the response was changed only slightly, the coefficients of `x1` and `x2` can move noticeably in opposite directions. This is the typical signature of approximate multicollinearity. The two predictors are so similar that the model has difficulty assigning the linear trend uniquely to one predictor rather than the other.

Next we center and scale the two predictors.

```
X_scaled <- model.matrix(y ~ scale(x1) + scale(x2), data = approx_data)
G_scaled <- crossprod(X_scaled)
lambda_scaled <- eigen(G_scaled, symmetric = TRUE)$values
condition_scaled <- condition_number_from_eigenvalues(lambda_scaled)
rank_scaled <- qr(X_scaled)$rank
```

```
G_scaled
```

```
##           (Intercept) scale(x1)   scale(x2)
## (Intercept) 1.000000e+01  0.000000 4.440892e-16
## scale(x1)   0.000000e+00  9.000000 8.999811e+00
## scale(x2)   4.440892e-16  8.999811 9.000000e+00
```

```
lambda_scaled
```

```
## [1] 1.799981e+01 1.000000e+01 1.889372e-04
```

```
condition_scaled
```

```
## [1] 95268.74
```

```
rank_scaled
```

```
## [1] 3
```

Centering and scaling change the numerical scale of the columns. This often improves the scale of the Gram matrix and makes computations easier to compare. However, centering and scaling do not create new independent information. If one predictor is exactly a linear combination of another, scaling will not remove exact multicollinearity. In the present approximate example, scaling improves the scale of the columns but the condition number can remain large because the two predictors remain almost the same variable.

3. Exercise 3 solution: multiple regression analysis in R

3.1 Part A: tibble and five basic dplyr verbs

The dataset `mtcars` is built into R. It contains fuel consumption and design information for 32 car models. The response variable `mpg` measures miles per gallon. Larger values indicate better fuel efficiency.

A tibble is a modern data frame. The pipe operator `|>` sends the object on its left into the function on its right. This makes a data analysis workflow readable from top to bottom.

```
car_data <- mtcars |>
  as_tibble(rownames = "car")
```

```
car_data
```

```
## # A tibble: 32 x 12
```

##	car	mpg	cyl	disp	hp	drat	wt	qsec	vs	am	gear	carb
##	<chr>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>
##	1 Mazda RX4	21	6	160	110	3.9	2.62	16.5	0	1	4	4
##	2 Mazda RX4 ~	21	6	160	110	3.9	2.88	17.0	0	1	4	4
##	3 Datsun 710	22.8	4	108	93	3.85	2.32	18.6	1	1	4	1
##	4 Hornet 4 D~	21.4	6	258	110	3.08	3.22	19.4	1	0	3	1
##	5 Hornet Spo~	18.7	8	360	175	3.15	3.44	17.0	0	0	3	2
##	6 Valiant	18.1	6	225	105	2.76	3.46	20.2	1	0	3	1
##	7 Duster 360	14.3	8	360	245	3.21	3.57	15.8	0	0	3	4
##	8 Merc 240D	24.4	4	147.	62	3.69	3.19	20	1	0	4	2
##	9 Merc 230	22.8	4	141.	95	3.92	3.15	22.9	1	0	4	2
##	10 Merc 280	19.2	6	168.	123	3.92	3.44	18.3	1	0	4	4
##	# i 22 more rows											

The five basic verbs are used as follows.

```

car_small <- car_data |>
  select(car, mpg, wt, hp, qsec, cyl, am)

car_filtered <- car_small |>
  filter(mpg >= 18)

car_mutated <- car_small |>
  mutate(
    hp_100 = hp / 100,
    am_label = if_else(am == 0, "automatic", "manual")
  )

car_sorted <- car_mutated |>
  arrange(desc(mpg))

car_summary <- car_mutated |>
  group_by(cyl) |>
  summarise(
    n = n(),
    mean_mpg = mean(mpg),
    sd_mpg = sd(mpg),
    .groups = "drop"
  )

car_small

```

```

## # A tibble: 32 x 7
##   car                mpg    wt    hp  qsec  cyl  am
##   <chr>            <dbl> <dbl> <dbl> <dbl> <dbl> <dbl>
## 1 Mazda RX4         21    2.62  110  16.5    6    1
## 2 Mazda RX4 Wag     21    2.88  110  17.0    6    1
## 3 Datsun 710        22.8  2.32   93  18.6    4    1
## 4 Hornet 4 Drive    21.4  3.22  110  19.4    6    0
## 5 Hornet Sportabout 18.7  3.44  175  17.0    8    0
## 6 Valiant           18.1  3.46  105  20.2    6    0
## 7 Duster 360        14.3  3.57  245  15.8    8    0
## 8 Merc 240D         24.4  3.19   62  20      4    0
## 9 Merc 230          22.8  3.15   95  22.9    4    0
## 10 Merc 280         19.2  3.44  123  18.3    6    0
## # i 22 more rows

```

```
car_filtered
```

```

## # A tibble: 19 x 7
##   car                mpg    wt    hp  qsec  cyl  am
##   <chr>            <dbl> <dbl> <dbl> <dbl> <dbl> <dbl>
## 1 Mazda RX4         21    2.62  110  16.5    6    1
## 2 Mazda RX4 Wag     21    2.88  110  17.0    6    1

```

```
## 3 Datsun 710      22.8  2.32   93  18.6    4    1
## 4 Hornet 4 Drive  21.4  3.22  110  19.4    6    0
## 5 Hornet Sportabout 18.7  3.44  175  17.0    8    0
## 6 Valiant        18.1  3.46  105  20.2    6    0
## 7 Merc 240D      24.4  3.19   62  20     4    0
## 8 Merc 230       22.8  3.15   95  22.9    4    0
## 9 Merc 280       19.2  3.44  123  18.3    6    0
## 10 Fiat 128      32.4  2.2    66  19.5    4    1
## 11 Honda Civic   30.4  1.62   52  18.5    4    1
## 12 Toyota Corolla 33.9  1.84   65  19.9    4    1
## 13 Toyota Corona 21.5  2.46   97  20.0    4    0
## 14 Pontiac Firebird 19.2  3.84  175  17.0    8    0
## 15 Fiat X1-9     27.3  1.94   66  18.9    4    1
## 16 Porsche 914-2  26    2.14   91  16.7    4    1
## 17 Lotus Europa  30.4  1.51  113  16.9    4    1
## 18 Ferrari Dino   19.7  2.77  175  15.5    6    1
## 19 Volvo 142E    21.4  2.78  109  18.6    4    1
```

```
car_mutated
```

```
## # A tibble: 32 x 9
```

```
##   car      mpg    wt    hp  qsec   cyl    am hp_100 am_label
##   <chr>    <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <chr>
## 1 Mazda RX4      21    2.62  110  16.5     6     1    1.1  manual
## 2 Mazda RX4 Wag   21    2.88  110  17.0     6     1    1.1  manual
## 3 Datsun 710     22.8    2.32   93  18.6     4     1    0.93 manual
## 4 Hornet 4 Drive  21.4    3.22  110  19.4     6     0    1.1  automatic
## 5 Hornet Sportabout 18.7    3.44  175  17.0     8     0    1.75 automatic
## 6 Valiant        18.1    3.46  105  20.2     6     0    1.05 automatic
## 7 Duster 360     14.3    3.57  245  15.8     8     0    2.45 automatic
## 8 Merc 240D      24.4    3.19   62  20      4     0    0.62 automatic
## 9 Merc 230       22.8    3.15   95  22.9     4     0    0.95 automatic
## 10 Merc 280      19.2    3.44  123  18.3     6     0    1.23 automatic
## # i 22 more rows
```

```
car_sorted
```

```
## # A tibble: 32 x 9
```

```
##   car      mpg    wt    hp  qsec   cyl    am hp_100 am_label
##   <chr>    <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <chr>
## 1 Toyota Corolla 33.9  1.84   65  19.9     4     1    0.65 manual
## 2 Fiat 128       32.4  2.2    66  19.5     4     1    0.66 manual
## 3 Honda Civic   30.4  1.62   52  18.5     4     1    0.52 manual
## 4 Lotus Europa  30.4  1.51  113  16.9     4     1    1.13 manual
## 5 Fiat X1-9     27.3  1.94   66  18.9     4     1    0.66 manual
## 6 Porsche 914-2  26    2.14   91  16.7     4     1    0.91 manual
## 7 Merc 240D      24.4    3.19   62  20      4     0    0.62 automatic
## 8 Datsun 710     22.8    2.32   93  18.6     4     1    0.93 manual
## 9 Merc 230       22.8    3.15   95  22.9     4     0    0.95 automatic
```

```
## 10 Toyota Corona    21.5  2.46    97  20.0     4     0  0.97 automatic
## # i 22 more rows
```

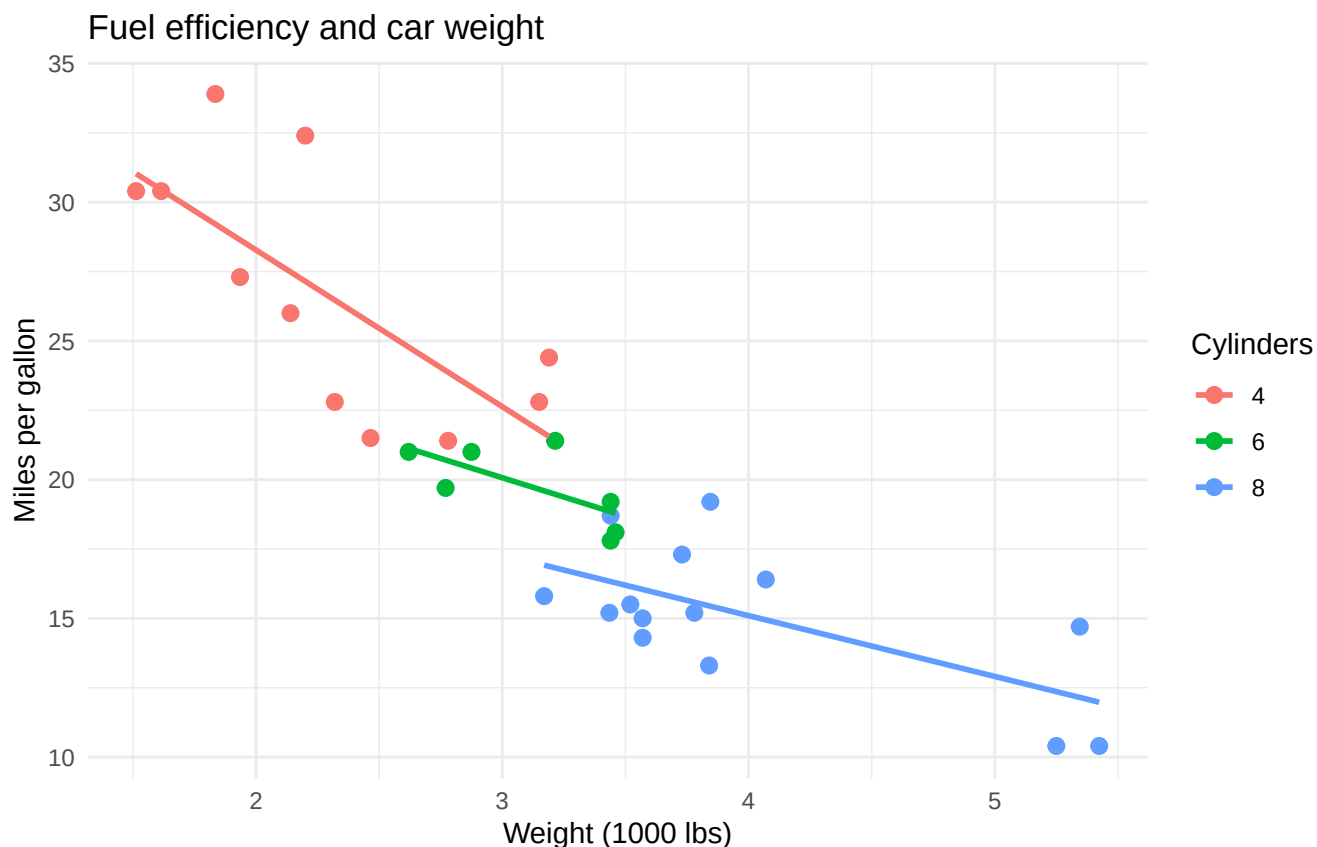
```
car_summary
```

```
## # A tibble: 3 x 4
##   cyl      n mean_mpg sd_mpg
##   <dbl> <int>   <dbl>  <dbl>
## 1     4    11    26.7   4.51
## 2     6     7    19.7   1.45
## 3     8    14    15.1   2.56
```

The grouped summary shows that cars with fewer cylinders tend to have higher fuel efficiency in this dataset. This is descriptive evidence, not yet a causal conclusion. Cylinder count is associated with other design variables such as weight and horsepower.

3.2 Part B: visual exploration with ggplot2

```
ggplot(car_data, aes(x = wt, y = mpg, color = factor(cyl))) +
  geom_point(size = 2.5) +
  geom_smooth(method = "lm", se = FALSE) +
  labs(
    x = "Weight (1000 lbs)",
    y = "Miles per gallon",
    color = "Cylinders",
    title = "Fuel efficiency and car weight"
  ) +
  theme_minimal()
```



The plot shows a clear negative association between weight and fuel efficiency: heavier cars tend to have lower miles per gallon. The color grouping also shows that cars with more cylinders tend to be heavier and less fuel efficient. The fitted lines are descriptive least-squares summaries of the observed data; they should not be interpreted as causal effects without a study design that justifies causal interpretation.

3.3 Part C: fit and interpret linear regression models

We fit the simple model

$$\text{mpg}_i = \beta_0 + \beta_1 \text{wt}_i + \varepsilon_i$$

and the multiple model

$$\text{mpg}_i = \beta_0 + \beta_1 \text{wt}_i + \beta_2 \text{hp}_i + \beta_3 \text{qsec}_i + \varepsilon_i.$$

```
fit_simple <- lm(mpg ~ wt, data = car_data)
fit_multiple <- lm(mpg ~ wt + hp + qsec, data = car_data)
```

```
summary(fit_simple)
```

```
##
## Call:
## lm(formula = mpg ~ wt, data = car_data)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
```

```
## -4.5432 -2.3647 -0.1252 1.4096 6.8727
##
## Coefficients:
##             Estimate Std. Error t value Pr(>|t|)
## (Intercept)  37.2851     1.8776  19.858 < 2e-16 ***
## wt          -5.3445     0.5591  -9.559 1.29e-10 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 3.046 on 30 degrees of freedom
## Multiple R-squared:  0.7528, Adjusted R-squared:  0.7446
## F-statistic: 91.38 on 1 and 30 DF, p-value: 1.294e-10
```

```
summary(fit_multiple)
```

```
##
## Call:
## lm(formula = mpg ~ wt + hp + qsec, data = car_data)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -3.8591 -1.6418 -0.4636  1.1940  5.6092
##
## Coefficients:
##             Estimate Std. Error t value Pr(>|t|)
## (Intercept)  27.61053     8.41993   3.279  0.00278 **
## wt          -4.35880     0.75270  -5.791 3.22e-06 ***
## hp          -0.01782     0.01498  -1.190  0.24418
## qsec         0.51083     0.43922   1.163  0.25463
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 2.578 on 28 degrees of freedom
## Multiple R-squared:  0.8348, Adjusted R-squared:  0.8171
## F-statistic: 47.15 on 3 and 28 DF, p-value: 4.506e-11
```

```
model_comparison <- tibble(
  model = c("Simple: mpg ~ wt", "Multiple: mpg ~ wt + hp + qsec"),
  residual_standard_error = c(summary(fit_simple)$sigma, summary(fit_multiple)$sigma),
  r_squared = c(summary(fit_simple)$r.squared, summary(fit_multiple)$r.squared),
  adjusted_r_squared = c(summary(fit_simple)$adj.r.squared, summary(fit_multiple)$adj.r.squared)
)
```

```
model_comparison
```

```
## # A tibble: 2 x 4
##   model                                residual_standard_er~1 r_squared adjusted_r_squared
##   <chr>                                <dbl>         <dbl>         <dbl>
## 1 Simple: mpg ~ wt                     3.05         0.753         0.745
```

```
## 2 Multiple: mpg ~ wt + hp +~           2.58      0.835           0.817
## # i abbreviated name: 1: residual_standard_error
```

```
coef(fit_simple)
```

```
## (Intercept)      wt
##   37.285126   -5.344472
```

```
coef(fit_multiple)
```

```
## (Intercept)      wt      hp      qsec
## 27.61052686 -4.35879720 -0.01782227  0.51083369
```

In the simple model, the estimated coefficient of `wt` is negative. This is practically reasonable: heavier cars tend to require more energy to move and therefore tend to have lower fuel efficiency.

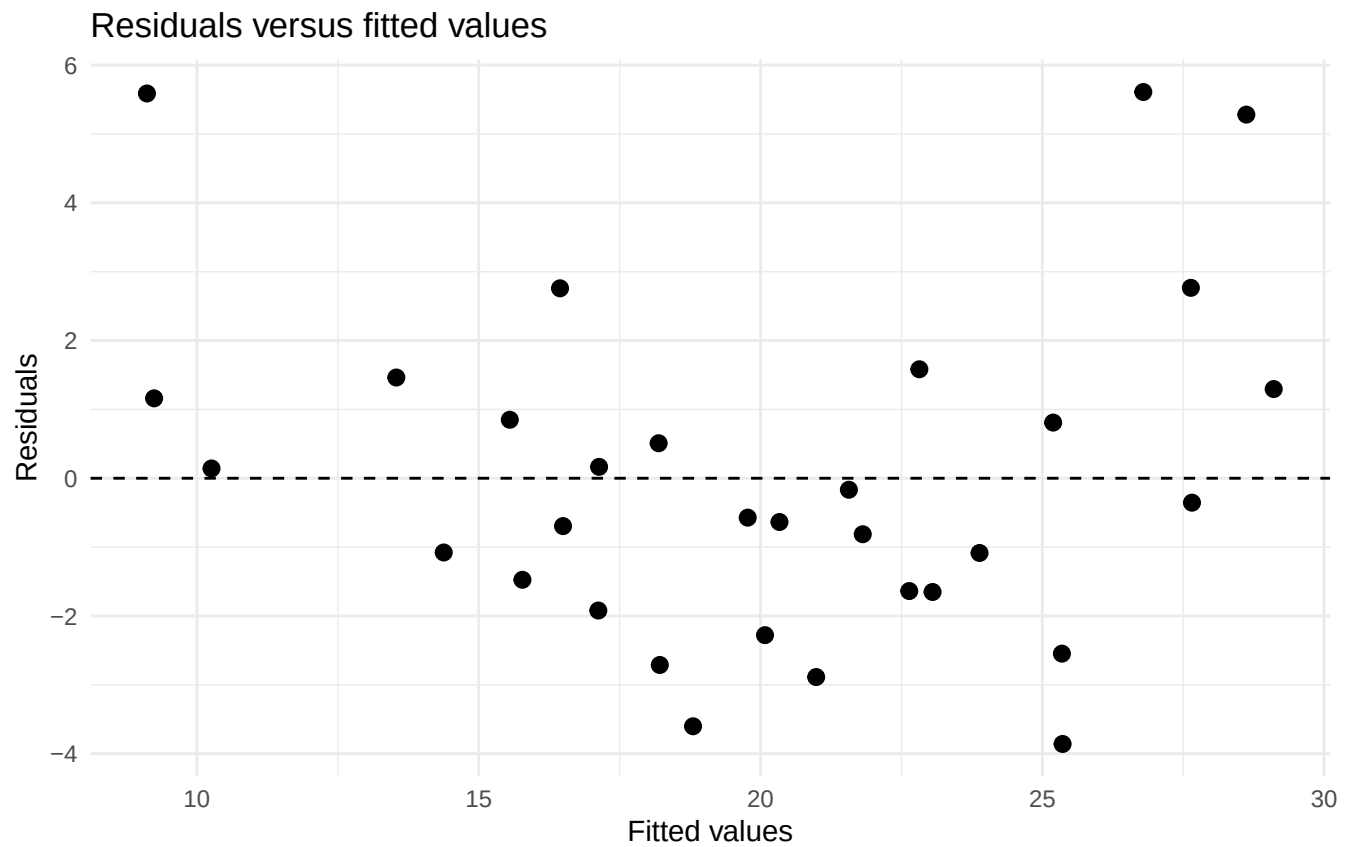
In the multiple model, the coefficient of `wt` is still negative. The coefficient of `hp` is also typically negative in this model, suggesting that, holding weight and quarter-mile time fixed, cars with greater horsepower tend to have lower fuel efficiency. The coefficient of `qsec` is interpreted conditionally: it describes the association between quarter-mile time and `mpg` after accounting for `wt` and `hp`. Because predictors in car design are strongly related to one another, these conditional interpretations should be made cautiously.

The multiple model has smaller residual standard error and larger adjusted R^2 than the simple model. Therefore it has better in-sample fit according to these summaries. However, this alone does not prove that it will predict future cars better. Training fit can improve when predictors are added even if the added predictors mainly fit sample-specific noise. Prediction should be assessed using validation data, test data, cross-validation, or a scientifically justified model-selection criterion.

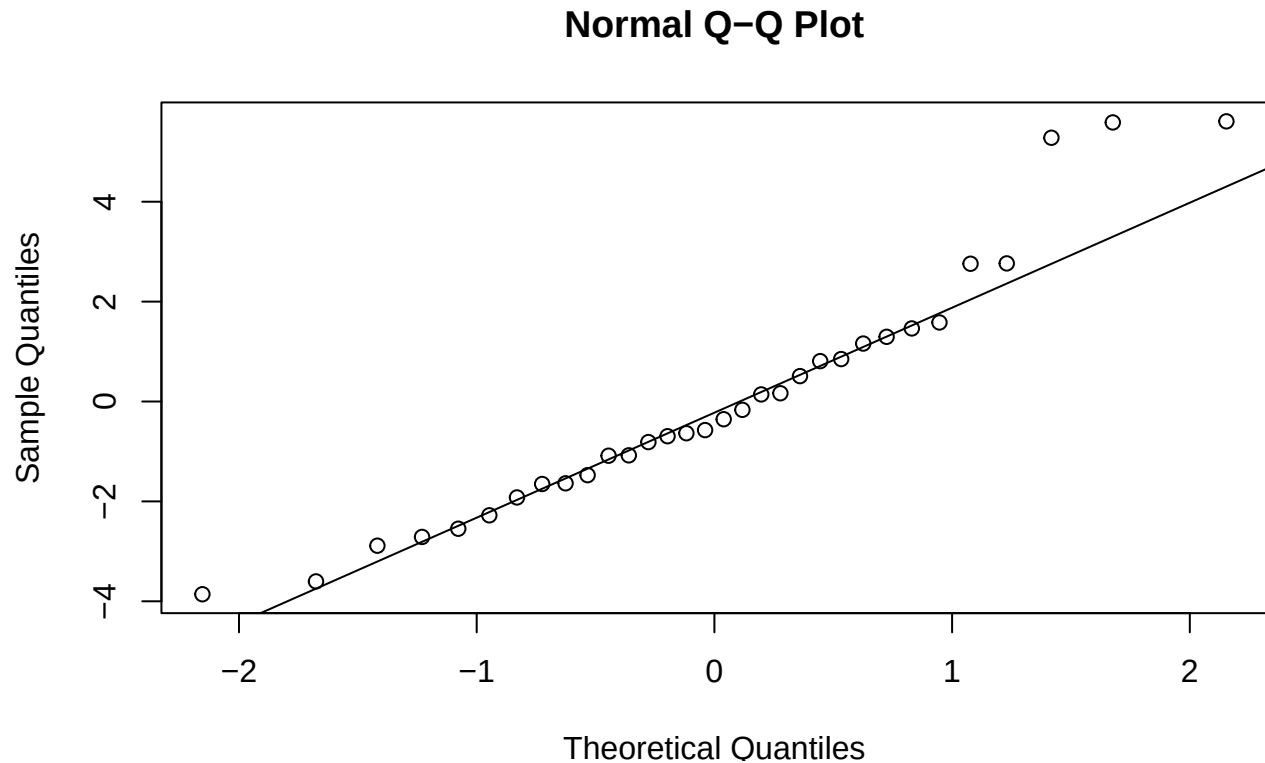
3.4 Part D: residual diagnostics

```
model_diagnostics <- tibble(
  fitted = fitted(fit_multiple),
  residual = resid(fit_multiple)
)

ggplot(model_diagnostics, aes(x = fitted, y = residual)) +
  geom_point(size = 2.5) +
  geom_hline(yintercept = 0, linetype = "dashed") +
  labs(
    x = "Fitted values",
    y = "Residuals",
    title = "Residuals versus fitted values"
  ) +
  theme_minimal()
```

```
qqnorm(resid(fit_multiple))  
qqline(resid(fit_multiple))
```



The residuals-versus-fitted plot is used to check whether residuals are centered around zero and whether their spread changes with the fitted value. Strong curvature would suggest lack of linearity, and a funnel shape would suggest nonconstant variance. The normal Q-Q plot checks whether the residual distribution is approximately normal. Since `mtcars` has only 32 observations, these plots should be interpreted cautiously. They are diagnostic tools, not formal proof that the model assumptions are true.

3.5 Part E: Gram matrix and condition number for the car model

We compute the design matrix, Gram matrix, eigenvalues, and condition number for the multiple regression model.

```
X_car <- model.matrix(fit_multiple)
G_car <- crossprod(X_car)
car_eigen_values <- eigen(G_car, symmetric = TRUE)$values
car_condition_number <- condition_number_from_eigenvalues(car_eigen_values)
```

```
G_car
```

```
##           (Intercept)           wt           hp           qsec
## (Intercept)      32.000      102.9520      4694.00      571.160
## wt              102.952       360.9011      16471.74      1828.095
## hp              4694.000      16471.7440      834278.00      81092.160
## qsec            571.160       1828.0946      81092.16      10293.480
```

```
car_eigen_values
```

```
## [1] 8.425354e+05 2.414634e+03 1.420920e+01 9.334865e-02
```

```

car_condition_number

## [1] 9025685

Now center and scale the predictors and recompute the same quantities.

X_car_scaled <- model.matrix(mpg ~ scale(wt) + scale(hp) + scale(qsec), data = car_data)
G_car_scaled <- crossprod(X_car_scaled)
car_scaled_eigen_values <- eigen(G_car_scaled, symmetric = TRUE)$values
car_scaled_condition_number <- condition_number_from_eigenvalues(car_scaled_eigen_values)

G_car_scaled

##           (Intercept)      scale(wt)      scale(hp)      scale(qsec)
## (Intercept)  3.200000e+01  2.053913e-15 -7.771561e-16  1.820766e-14
## scale(wt)    2.053913e-15  3.100000e+01  2.042118e+01 -5.416192e+00
## scale(hp)   -7.771561e-16  2.042118e+01  3.100000e+01 -2.195493e+01
## scale(qsec)  1.820766e-14 -5.416192e+00 -2.195493e+01  3.100000e+01

car_scaled_eigen_values

## [1] 63.808661 32.000000 25.598964  3.592375

car_scaled_condition_number

## [1] 17.76225

```

The raw design matrix usually has a much larger condition number than the centered-and-scaled design matrix. This happens because the raw predictors are measured in very different units: `wt` is in thousands of pounds, `hp` is horsepower, and `qsec` is time in seconds. The intercept column can also interact numerically with predictors whose sample means are far from zero. Centering removes the sample mean of each predictor, and scaling places predictors on comparable numerical scales.

Centering and scaling do not change the essential fitted values when the model includes an intercept and the predictors are transformed by invertible linear rescalings. To see this, suppose a predictor z is replaced by

$$z^* = \frac{z - \bar{z}}{s_z},$$

where $s_z > 0$. A fitted model

$$a^* + b^* z^*$$

can be rewritten in the original coordinate as

$$a^* + b^* \frac{z - \bar{z}}{s_z} = \left(a^* - \frac{b^* \bar{z}}{s_z} \right) + \left(\frac{b^*}{s_z} \right) z.$$

Thus the set of possible fitted lines is the same; only the coefficient parameterization changes.

A large condition number means that at least one direction in coefficient space is weakly identified by the design. Therefore we should be cautious when interpreting individual coefficients. The fitted values may be reasonably stable while individual coefficients are unstable, especially when predictors are strongly correlated or nearly linearly dependent.

4. Some remarks

For the theoretical exercises above, we use only finite-dimensional Euclidean linear algebra. The centering result depends on the orthogonal decomposition

$$\mathbb{R}^n = \text{span}(1_n) \oplus \{u \in \mathbb{R}^n : 1_n^\top u = 0\}.$$

The Gram-matrix result depends on the identity

$$v^\top X^\top X v = \|Xv\|_2^2.$$